

Smart contract audit

rain.lib.leakybucket

Content

Project description	3
Executive summary	3
Reviews	3
Technical analysis and findings	4
Security findings	5
**** Critical	5
*** High	5
** Medium	5
* Low	5
L01 - Leak rate changes apply retroactively to the time since the last checkpoint	5
Informational	6
I01 - Documented checkpoint operation always reverts	6
I02 - Overflow explanation reverses the effect on headroom	6
General Risks and Assumptions	7
Approach and methodology	8



Project description

The rain.lib.leakybucket repository implements a Solidity library for enforcing rate limits in EVM-compatible smart contracts, with a primary focus on capping token minting. It uses a leaky bucket model with a configurable capacity that limits burst size and a per-second leak rate that determines how quickly available allowance recovers. Unused time cannot accumulate allowance beyond the configured capacity.

The library provides pure functions for checking available allowance and calculating updated bucket state, leaving storage, access control, and governance to the integrating contract. It packs the bucket level and timestamp into a single checkpoint word, allowing each successful fill to be recorded with one storage write. Saturating arithmetic and explicit bounds checks handle arithmetic extremes and backward timestamps, while Foundry tests cover capacity limits, state transitions, and integration behavior.

Executive summary

Type	Library
Languages	Solidity
Methods	Architecture Review, Manual Review, Unit Testing, Functional Testing, Automated Review
Documentation	README.md
Repositories	https://github.com/rainlanguage/rain.lib.leakybucket

Reviews

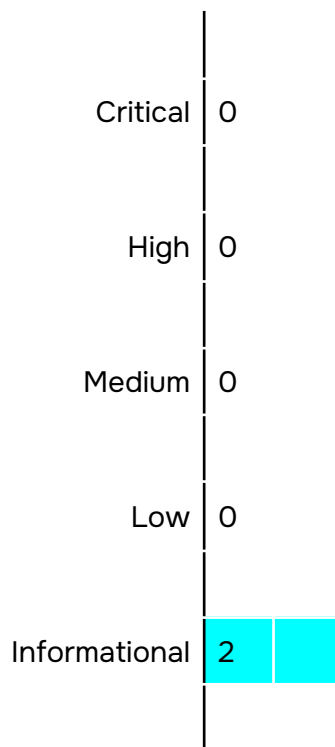
Date	Repository	Commit
09/02/26	rain.lib.leakybucket	06baa59b15b0b4ad51189288e3137eee677db37d tag sol-v0.4.0

Scope

Contracts
src/lib/LibLeakyBucket.sol



Technical analysis and findings



Security findings

**** Critical

No critical severity issue found.

*** High

No high severity issue found.

** Medium

No medium severity issue found.

* Low

L01 - Leak rate changes apply retroactively to the time since the last checkpoint

The `leak` is computed at read time as `(timestamp - checkpointTimestamp) * bucket.leakRate`, always using the current `leakRate`. The checkpoint records only the level and its second, not the rate that was in force. So any change to `leakRate` re-rates the whole interval since the last fill, not only the time after the change.

Raising the rate hands out immediate headroom that was never earned under either policy. With `capacity = 100` and `leakRate = 1/h`, a minter fills to 100 at 00:00. At 10:00 the headroom is 10. Governance then raises `leakRate` to 100/h. At that same second the headroom reads 100, not 10, and the minter can mint 200 in 10 hours where the policies allowed 110. For the same reason a "pause" via `leakRate = 0` is not a pause: credit keeps building while the rate is zero, and restoring the rate drains the bucket at once, freeing a full `capacity`.

The excess is bounded by the outstanding level, so it never exceeds one `capacity` and the burst bound holds. The README makes this worse by telling integrators to "write the two policy fields and leave the checkpoint alone" (the policy changed, the bucket's history did not). The library has no way left to settle a bucket at the old rate before a change.

Path: `src/lib/LibLeakyBucket.sol`, `README.md`

Recommendation: State in the NatSpec and README that `leakRate` changes apply retroactively, and document a settle-before-change procedure: read the current level at the old rate, store `pack(level, max(timestamp, storedTimestamp))`, then write the new rate. Consider exporting a helper that does this (e.g. `settle(bucket, timestamp)`), and add a `setLeakRate` action to the invariant handler.



Found in: 06baa59b

Status: New

Informational

I01 - Documented checkpoint operation always reverts

The README describes `fill(bucket, timestamp, 0)` as a checkpoint-only operation, but `fill` explicitly rejects zero amounts. Its claim that the amount returned by `headroomAt` always fits also excludes the zero-headroom case.

Path: README.md, src/lib/LibLeakyBucket.sol

Recommendation: Replace the example with a supported checkpoint procedure and document that only positive fills are accepted.

Found in: 06baa59b

Status: New

I02 - Overflow explanation reverses the effect on headroom

README says a wrapped `elapsed * leakRate` is "a small leak, which is free headroom". Backwards: a small leak keeps the level high, so it yields less headroom (e.g. level 995 → headroom 2605) than saturation (level 0 → headroom 3600), which is the mathematically exact answer since any overflowing product exceeds every storable level.

Path: README.md

Recommendation: Reword to exactness: saturation preserves the exact drained-to-zero result because an overflowing product exceeds any storable level.

Found in: 06baa59b

Status: New



General Risks and Assumptions

- Correct integration and state persistence. The library relies on the integrating contract to enforce rate limits on every relevant operation and persist the checkpoint returned by fill. Bypassing enforcement, failing to store updates, or resetting checkpoints can undermine the limits. Mint authorization and access control remain the caller's responsibility.
- Trusted governance and appropriate policy configuration. Capacity and leak rate must reflect acceptable burst exposure and sustained throughput. A compromised authorized minter can consume the full available capacity and continue at the configured leak rate. Policy changes require appropriate access controls; the current leak rate applies to the entire interval since the stored checkpoint, so changing it can retroactively affect available allowance.
- Reliable timestamps and consistent units. The library assumes callers provide timestamps in seconds and express capacity, fill amounts, and leak rate in consistent units. Incorrect or artificially advanced timestamps can restore allowance prematurely. Backward timestamps receive no leakage credit, but the library cannot verify the authenticity of caller-supplied time.



Approach and methodology

To establish a uniform evaluation, we define the following terminology in accordance with the OWASP Risk Rating

Methodology:

	Likelihood Indicates the probability of a specific vulnerability being discovered and exploited in real-world scenarios
	Impact Measures the technical loss and business repercussions resulting from a successful attack
	Severity Reflects the comprehensive magnitude of the risk, combining both the probability of occurrence (likelihood) and the extent of potential consequences (impact)

Likelihood and impact are divided into three levels: High H, Medium M, and Low L. The severity of a risk is a blend of these two factors, leading to its classification into one of four tiers: Critical, High, Medium, or Low.

When we identify an issue, our approach may include deploying contracts on our private testnet for validation through testing. Where necessary, we might also create a Proof of Concept PoC to demonstrate potential exploitability. In particular, we perform the audit according to the following procedure:

	Advanced DeFi Scrutiny We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs
	Semantic Consistency Checks We then manually check the logic of implemented smart contracts and compare with the description in the white paper.
	Security Analysis The process begins with a comprehensive examination of the system to gain a deep understanding of its internal mechanisms, identifying any irregularities and potential weak spots.